

Laboration 8 – Händelser, timers och återanslutning

Uppgift

Utöka ett MQTT-flöde med minst två händelsekällor, ett synligt anslutningstillstånd och räknare. Börja med simulatorn. Fortsätt på ESP32-C6 när grundflödet är förstått och miljön är förberedd.

Git är valfritt för den fristående simulatorlaborationen. Ändringar i IoT-caset sparas i Git.

Del 1 – Trådar, mutex och join

Läs [demo_threads/pthread_demo.c](#). Markera:

- Huvudtråden
- De två arbetstrådarna
- Data som delas mellan trådarna
- Den kritiska sektionen
- Punkten där huvudtråden väntar

Bygg och kör:

```
cd dag_8/demo_threads
cc -std=c11 -Wall -Wextra -Wpedantic -pthread pthread_demo.c -o
pthread_demo
./pthread_demo
```

Kör minst tre gånger. Notera om ordningen mellan **WORKER**-raderna varierar och förklara varför slutvärdet ändå blir stabilt. På Windows utan WSL används Win32-varianten enligt [demots README](#).

Del 2 – Förutsäg simulatorn

Läs intervallen och avbrottstiderna i [demo_simulator/simulator.cpp](#). Fyll i:

- När temperaturhändelser skapas
- När luftfuktighetshändelser skapas
- Vilka händelser som inträffar under avbrottet
- Vilken publicering som blir den första efter återanslutning

Del 3 – Bygg och kör simulatorn

```
cd dag_8/demo_simulator
g++ -std=c++17 -Wall -Wextra -Wpedantic simulator.cpp -o simulator
./simulator
```

Fyll i [händelseprotokollet](#). Kontrollera att summan av publicerade och tappade motsvarar skapade händelser.

Del 4 – Lägg till en händelse

Lägg till exempelvis en larmhändelse var tionde sekund. Ange eget `sensor_id`, värde och enhet. Uppdatera programmets förväntade sluträknare och kontrollera att exitkoden fortfarande blir 0.

Del 5 – Jämför köpolicy

Simulatorn tappar händelser under avbrott. Beskriv hur resultatet skulle ändras om systemet i stället:

- Behöll alla händelser i RAM
- Behöll endast senaste värde per sensor
- Sparade kritiska händelser i flash

Välj en policy för två olika datatyper och motivera skillnaden.

Del 6 – ESP32-C6

Följ [ESP32-demots README](#). Konfigurera inte verkliga lösenord i en fil som ska delas. Börja med bygge. Flasha först när rätt kort och port har identifierats.

Verifiera:

- Fyra `TASK_CREATED` med förväntade namn och prioriteringar
- Två timers med olika intervall och korta callbacks
- Två sensortasks som väcks av var sin timer
- Sensorproducenter som lämnar event till samma kö
- Wi-Fi- och MQTT-event som uppdaterar anslutningstillståndet
- `MQTT_EVENT_CONNECTED` i seriell logg
- Mottagna MQTTS-meddelanden hos subscribern
- Frånkoppling när brokern stoppas
- Ny publicering efter att brokern startats igen

Om fysisk körning inte är möjlig granskar gruppen koden, markerar ansvaren och använder simulatorns körbevis. Dokumentera begränsningen.

Valbar fördjupning

Välj en:

- Behåll senaste värde under avbrott och publicera det efter återanslutning
- Lägg till en begränsad RAM-kö och mät ködjup
- Använd QoS 1 och jämför vad en accepterad publish betyder mot en brokerkvittens
- Inför exponentiell backoff i en egen klient där automatisk återanslutning är avstängd
- Lägg till en tredje sensortask och skicka dess event genom samma kö

Arbetsresultat

- Förklaring av create, mutex och join i datordemot

- Minst två händelsekällor eller sensortasks
- Dokumenterat anslutningstillstånd
- Räknare som går att kontrollera
- Ett avbrott och en verifierad återanslutning
- Motiverad policy för händelser under avbrott